JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class WeekFields

java.lang.Object
    java.time.temporal.WeekFields

All Implemented Interfaces:

`Serializable`

```
public final class WeekFields
extends Object
implements Serializable
```

Localized definitions of the day-of-week, week-of-month and week-of-year fields.

A standard week is seven days long, but cultures have different definitions for some other aspects of a week. This class represents the definition of the week, for the purpose of providing `TemporalField` instances.

WeekFields provides five fields, `dayOfWeek()`, `weekOfMonth()`, `weekOfYear()`, `weekOfWeekBasedYear()`, and `weekBasedYear()` that provide access to the values from any temporal object.

The computations for day-of-week, week-of-month, and week-of-year are based on the proleptic-year, month-of-year, day-of-month, and ISO day-of-week which are based on the epoch-day and the chronology. The values may not be aligned with the year-of-Era depending on the Chronology.

A week is defined by:

- The first day-of-week. For example, the ISO-8601 standard considers Monday to be the first day-of-week.
- The minimal number of days in the first week. For example, the ISO-8601 standard counts the first week as needing at least 4 days.

Together these two values allow a year or month to be divided into weeks.

Week of Month

One field is used: week-of-month. The calculation ensures that weeks never overlap a month boundary. The month is divided into periods where each period starts on the defined first day-of-week. The earliest period is referred to as week 0 if it has less than the minimal number of days and week 1 if it has at least the minimal number of days.

Examples of WeekFields

Date	Day-of-week	First day: Monday Minimal days: 4	First day: Monday Minimal days: 5
2008-12-31	Wednesday	Week 5 of December 2008	Week 5 of December 2008
2009-01-01	Thursday	Week 1 of January 2009	Week 0 of January 2009
2009-01-04	Sunday	Week 1 of January 2009	Week 0 of January 2009
2009-01-05	Monday	Week 2 of January 2009	Week 1 of January 2009

Week of Year

One field is used: week-of-year. The calculation ensures that weeks never overlap a year boundary. The year is divided into periods where each period starts on the defined first day-of-week. The earliest period is referred to as week 0 if it has less than the minimal number of days and week 1 if it has at least the minimal number of days.

Week Based Year

Two fields are used for week-based-year, one for the `week-of-week-based-year` and one for `week-based-year`. In a week-based-year, each week belongs to only a single year. Week 1 of a year is the first week that starts on the first day-of-week and has at least the minimum number of days. The first and last weeks of a year may contain days from the previous calendar year or next calendar year respectively.

Examples of WeekFields for week-based-year

Date	Day-of-week	First day: Monday Minimal days: 4	First day: Monday Minimal days: 5
2008-12-31	Wednesday	Week 1 of 2009	Week 53 of 2008
2009-01-01	Thursday	Week 1 of 2009	Week 53 of 2008
2009-01-04	Sunday	Week 1 of 2009	Week 53 of 2008
2009-01-05	Monday	Week 2 of 2009	Week 1 of 2009

Implementation Requirements:

This class is immutable and thread-safe.

Since:

1.8

See Also:

Serialized Form

Field Summary

Fields

Modifier and Type	Field	Description
static final WeekFields	ISO	The ISO-8601 definition, where a week starts on Monday and the first week has a minimum of 4 days.
static final WeekFields	SUNDAY_START	The common definition of a week that starts on Sunday and the first week has a minimum of 1 day.
static final TemporalUnit	WEEK_BASED_YEARS	The unit that represents week-based-years for the purpose of addition and subtraction.

Method Summary

All Methods | Static Methods

Modifier and Type	Method	Description
TemporalField	dayOfWeek()	Returns a field to access the day of week based on this WeekFields.
boolean	equals(Object object)	Checks if this WeekFields is equal to the specified object.
DayOfWeek	getFirstDayOfWeek()	Gets the first day-of-week.
int	getMinimalDaysInFirstWee	Gets the minimal number of days in the first week.
int	hashCode()	A hash code for this WeekFields.
static WeekFields	of (DayOfWeek firstDayOfWee int minimalDaysInFirstWe	Obtains an instance of WeekFields from the

		first day-of-week and minimal days.
static WeekFields	**of**(Locale locale)	Obtains an instance of WeekFields appropriate for a locale.
String	**toString**()	A string representation of this WeekFields instance.
TemporalField	**weekBasedYear**()	Returns a field to access the year of a week-based-year based on this WeekFields.
TemporalField	**weekOfMonth**()	Returns a field to access the week of month based on this WeekFields.
TemporalField	**weekOfWeekBasedYear**()	Returns a field to access the week of a week-based-year based on this WeekFields.
TemporalField	**weekOfYear**()	Returns a field to access the week of year based on this WeekFields.

Methods declared in class Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Field Details

ISO

```
public static final WeekFields ISO
```

The ISO-8601 definition, where a week starts on Monday and the first week has a minimum of 4 days.

The ISO-8601 standard defines a calendar system based on weeks. It uses the week-based-year and week-of-week-based-year concepts to split up the passage of days instead of the standard year/month/day.

Note that the first week may start in the previous calendar year. Note also that the first few days of a calendar year may be in the week-based-year corresponding to the previous calendar year.

SUNDAY_START

```
public static final WeekFields SUNDAY_START
```

The common definition of a week that starts on Sunday and the first week has a minimum of 1 day.

Defined as starting on Sunday and with a minimum of 1 day in the month. This week definition is in use in the US and other European countries.

WEEK_BASED_YEARS

```
public static final TemporalUnit WEEK_BASED_YEARS
```

The unit that represents week-based-years for the purpose of addition and subtraction.

This allows a number of week-based-years to be added to, or subtracted from, a date. The unit is equal to either 52 or 53 weeks. The estimated duration of a week-based-year is the same as that of a standard ISO year at `365.2425 Days`.

The rules for addition add the number of week-based-years to the existing value for the week-based-year field retaining the week-of-week-based-year and day-of-week, unless the week number it too large for the target year. In that case, the week is set to the last week of the year with the same day-of-week.

This unit is an immutable and thread-safe singleton.

Method Details

of

```
public static WeekFields of(Locale locale)
```

Obtains an instance of `WeekFields` appropriate for a locale.

This will look up appropriate values from the provider of localization data. If the locale contains "fw" (First day of week) and/or "rg" (Region Override) Unicode extensions, returned instance will reflect the values specified with

those extensions. If both "fw" and "rg" are specified, the value from the "fw" extension supersedes the implicit one from the "rg" extension.

For example, users who are interested in using an English locale, but want the
first day of the week that corresponds with the ISO-8601 standard can call

```
Locale enIsoLoc = Locale.forLanguageTag("en-u-fw-mon");
WeekFields.of(enIsoLoc).getFirstDayOfWeek(); // returns MONDAY
```

Parameters:

`locale` - the locale to use, not null

Returns:

the week-definition, not null

of

```
public static WeekFields of(DayOfWeek firstDayOfWeek,
                            int minimalDaysInFirstWeek)
```

Obtains an instance of `WeekFields` from the first day-of-week and minimal days.

The first day-of-week defines the ISO `DayOfWeek` that is day 1 of the week. The minimal number of days in the first week defines how many days must be present in a month or year, starting from the first day-of-week, before the week is counted as the first week. A value of 1 will count the first day of the month or year as part of the first week, whereas a value of 7 will require the whole seven days to be in the new month or year.

WeekFields instances are singletons; for each unique combination of `firstDayOfWeek` and `minimalDaysInFirstWeek` the same instance will be returned.

Parameters:

`firstDayOfWeek` - the first day of the week, not null

`minimalDaysInFirstWeek` - the minimal number of days in the first week, from 1 to 7

Returns:

the week-definition, not null

Throws:

`IllegalArgumentException` - if the minimal days value is less than one or greater than 7

getFirstDayOfWeek

```
public DayOfWeek getFirstDayOfWeek()
```

Gets the first day-of-week.

The first day-of-week varies by culture. For example, the US uses Sunday, while France and the ISO-8601 standard use Monday. This method returns the first day using the standard `DayOfWeek` enum.

Returns:

the first day-of-week, not null

getMinimalDaysInFirstWeek

```
public int getMinimalDaysInFirstWeek()
```

Gets the minimal number of days in the first week.

The number of days considered to define the first week of a month or year varies by culture. For example, the ISO-8601 requires 4 days (more than half a week) to be present before counting the first week.

Returns:

the minimal number of days in the first week of a month or year, from 1 to 7

dayOfWeek

```
public TemporalField dayOfWeek()
```

Returns a field to access the day of week based on this `WeekFields`.

This is similar to `ChronoField.DAY_OF_WEEK` but uses values for the day-of-week based on this `WeekFields`.
The days are numbered from 1 to 7 where the `first day-of-week` is assigned the value 1.

For example, if the first day-of-week is Sunday, then that will have the value 1, with other days ranging from Monday as 2 to Saturday as 7.

In the resolving phase of parsing, a localized day-of-week will be converted to a standardized `ChronoField` day-of-week. The day-of-week must be in the valid range 1 to 7. Other fields in this class build dates using the standardized day-of-week.

Returns:

a field providing access to the day-of-week with localized numbering, not null

weekOfMonth

```
public TemporalField weekOfMonth()
```

Returns a field to access the week of month based on this `WeekFields`.

This represents the concept of the count of weeks within the month where weeks start on a fixed day-of-week, such as Monday. This field is typically used with `dayOfWeek()`.

Week one (1) is the week starting on the `getFirstDayOfWeek()` where there are at least `getMinimalDaysInFirstWeek()` days in the month. Thus, week one may start up to `minDays` days before the start of the month. If the first week starts after the start of the month then the period before is week zero (0).

For example:
- if the 1st day of the month is a Monday, week one starts on the 1st and there is no week zero
- if the 2nd day of the month is a Monday, week one starts on the 2nd and the 1st is in week zero
- if the 4th day of the month is a Monday, week one starts on the 4th and the 1st to 3rd is in week zero
- if the 5th day of the month is a Monday, week two starts on the 5th and the 1st to 4th is in week one

This field can be used with any calendar system.

In the resolving phase of parsing, a date can be created from a year, week-of-month, month-of-year and day-of-week.

In strict mode, all four fields are validated against their range of valid values. The week-of-month field is validated to ensure that the resulting month is the month requested.

In smart mode, all four fields are validated against their range of valid values. The week-of-month field is validated from 0 to 6, meaning that the resulting date can be in a different month to that specified.

In lenient mode, the year and day-of-week are validated against the range of valid values. The resulting date is calculated equivalent to the following four stage approach. First, create a date on the first day of the first week of January in the requested year. Then take the month-of-year, subtract one, and add the amount in months to the date. Then take the week-of-month, subtract one, and add the amount in weeks to the date. Finally, adjust to the correct day-of-week within the localized week.

Returns:

a field providing access to the week-of-month, not null

weekOfYear

```
public TemporalField weekOfYear()
```

Returns a field to access the week of year based on this `WeekFields`.

This represents the concept of the count of weeks within the year where weeks start on a fixed day-of-week, such as Monday. This field is typically used with `dayOfWeek()`.

Week one(1) is the week starting on the `getFirstDayOfWeek()` where there are at least `getMinimalDaysInFirstWeek()` days in the year. Thus, week one may start up to `minDays` days before the start of the year. If the first week starts after the start of the year then the period before is week zero (0).

For example:
- if the 1st day of the year is a Monday, week one starts on the 1st and there is no week zero
- if the 2nd day of the year is a Monday, week one starts on the 2nd and the 1st is in week zero
- if the 4th day of the year is a Monday, week one starts on the 4th and the 1st to 3rd is in week zero
- if the 5th day of the year is a Monday, week two starts on the 5th and the 1st to 4th is in week one

This field can be used with any calendar system.

In the resolving phase of parsing, a date can be created from a year, week-of-year and day-of-week.

In strict mode, all three fields are validated against their range of valid values. The week-of-year field is validated to ensure that the resulting year is the year requested.

In smart mode, all three fields are validated against their range of valid values. The week-of-year field is validated from 0 to 54, meaning that the resulting date can be in a different year to that specified.

In lenient mode, the year and day-of-week are validated against the range of valid values. The resulting date is calculated equivalent to the following three stage approach. First, create a date on the first day of the first week in the requested year. Then take the week-of-year, subtract one, and add the amount in weeks to the date. Finally, adjust to the correct day-of-week within the localized week.

Returns:

a field providing access to the week-of-year, not null

weekOfWeekBasedYear

```
public TemporalField weekOfWeekBasedYear()
```

Returns a field to access the week of a week-based-year based on this `WeekFields`.

This represents the concept of the count of weeks within the year where weeks start on a fixed day-of-week, such as Monday and each week belongs to exactly one year. This field is typically used with `dayOfWeek()` and `weekBasedYear()`.

Week one(1) is the week starting on the `getFirstDayOfWeek()` where there are at least `getMinimalDaysInFirstWeek()` days in the year. If the first week starts after the start of the year then the period before is in the last week of the previous year.

For example:
- if the 1st day of the year is a Monday, week one starts on the 1st
- if the 2nd day of the year is a Monday, week one starts on the 2nd and the 1st is in the last week of the previous year
- if the 4th day of the year is a Monday, week one starts on the 4th and the 1st to 3rd is in the last week of the previous year
- if the 5th day of the year is a Monday, week two starts on the 5th and the 1st to 4th is in week one

This field can be used with any calendar system.

In the resolving phase of parsing, a date can be created from a week-based-year, week-of-year and day-of-week.

In strict mode, all three fields are validated against their range of valid values. The week-of-year field is validated to ensure that the resulting week-based-year is the week-based-year requested.

In smart mode, all three fields are validated against their range of valid values. The week-of-week-based-year field is validated from 1 to 53, meaning that the resulting date can be in the following week-based-year to that specified.

In lenient mode, the year and day-of-week are validated against the range of valid values. The resulting date is calculated equivalent to the following three stage approach. First, create a date on the first day of the first week in the requested week-based-year. Then take the week-of-week-based-year, subtract one, and add the amount in weeks to the date. Finally, adjust to the correct day-of-week within the localized week.

Returns:

a field providing access to the week-of-week-based-year, not null

weekBasedYear

```
public TemporalField weekBasedYear()
```

Returns a field to access the year of a week-based-year based on this `WeekFields`.

This represents the concept of the year where weeks start on a fixed day-of-week, such as Monday and each week belongs to exactly one year. This field is typically used with `dayOfWeek()` and `weekOfWeekBasedYear()`.

Week one(1) is the week starting on the `getFirstDayOfWeek()` where there are at least `getMinimalDaysInFirstWeek()` days in the year. Thus, week one may start before the start of the year. If the first week starts after the start of the year then the period before is in the last week of the previous year.

This field can be used with any calendar system.

In the resolving phase of parsing, a date can be created from a week-based-year, week-of-year and day-of-week.

In strict mode, all three fields are validated against their range of valid values. The week-of-year field is validated to ensure that the resulting week-based-year is the week-based-year requested.

In smart mode, all three fields are validated against their range of valid values. The week-of-week-based-year field is validated from 1 to 53, meaning that the resulting date can be in the following week-based-year to that specified.

In lenient mode, the year and day-of-week are validated against the range of valid values. The resulting date is calculated equivalent to the following three stage approach. First, create a date on the first day of the first week in the requested week-based-year. Then take the week-of-week-based-year, subtract one, and add the amount in weeks to the date. Finally, adjust to the correct day-of-week within the localized week.

Returns:

a field providing access to the week-based-year, not null

equals

```
public boolean equals(Object object)
```

Checks if this `WeekFields` is equal to the specified object.

The comparison is based on the entire state of the rules, which is the first day-of-week and minimal days.

Overrides:

`equals` in class `Object`

Parameters:

`object` - the other rules to compare to, null returns false

Returns:

true if this is equal to the specified rules

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

A hash code for this `WeekFields`.

Overrides:

`hashCode` in class `Object`

Returns:

a suitable hash code

See Also:

`Object.equals(java.lang.Object)`,
`System.identityHashCode(Object)`

toString

```
public String toString()
```

A string representation of this `WeekFields` instance.

Overrides:

`toString` in class `Object`

Returns:

the string representation, not null

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA

94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.